
Group 12

Group 12

Software Requirement Specification

For SinhaSpeech

Version 1.0

Project ID : P13

Group ID : 12

Team Members:

230283A Yohan Jayasinghe

230295L Tharupahan Jayawardana

230277J Imindu Jayasekara

Mentor: Dr. Buddhika Karunarathna

Teaching Assistant: Mr. Ilampoornan

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Table of Content

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions, Acronyms, and Abbreviations.....	4
1.4 References.....	5
1.5 Overview.....	6
2. Overall Description.....	6
3. Specific Requirements.....	7
3.1 Functionality.....	7
3.1.1 Audio preprocessing pipeline.....	7
3.1.2 Model Fine Tuning.....	7
3.1.3 Model Evaluation and Benchmarking.....	8
3.1.4 Classroom Captioning.....	8
3.1.5 Transcript Editing, Export, and Confidence Highlighting.....	8
3.2 Usability.....	9
3.3 Reliability.....	9
3.4 Performance and Security.....	10
3.5 Supportability.....	11
3.6 Design Constraints.....	11
3.7 On-line User Documentation and Help System Requirements.....	12
3.8 Purchased Components.....	12
3.9 Interfaces.....	12
3.9.1 User Interfaces.....	12
3.9.2 Hardware Interfaces.....	13
3.9.3 Software Interfaces.....	13
3.9.4 Communications Interfaces.....	14
3.10 Licensing, Legal, Copyright, and Other Notices.....	14
3.11 Applicable Standards.....	15
4. Supporting Information.....	15
4.1 References.....	15

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Revision History

Date	Version	Description	Author
30-July-2026	1.0	Initial Document	Group 12

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

1. Introduction

This Software Requirements Specification (SRS) outlines the design and functionalities of SinhaSpeech, a Sinhala speech-to-text Automatic Speech Recognition (ASR) system developed by Group 12 for In22-S5-CS3501 Data Science and Engineering Project, Department of Computer Science and Engineering, University of Moratuwa. The system fine-tunes OpenAI's multilingual Whisper model on curated Sinhala speech corpora to produce a general-purpose Sinhala ASR model that is measurably more accurate than the fine-tunes currently available, and integrates the resulting model into an accessibility application that helps Deaf and hard-of-hearing students and students with writing difficulties follow lessons and complete their work more easily. This document covers both the guaranteed core scope of the project (Phase 1: fine-tuning and benchmarking the Sinhala Whisper model) and its committed extension (Phase 2: the accessibility application built on the Phase 1 model).

1.1 Purpose

Sinhala speech recognition still lags behind high-resource languages like English. The Whisper fine-tunes currently available for Sinhala are general-purpose, aimed at everyday dictation or news transcription, and are not accurate enough to be relied on by other systems built on top of them. Before any more specialised Sinhala speech application can be built, there needs to be a Sinhala ASR model that is actually accurate enough to use, not just roughly workable. This document describes the plan for the system we aim to build: what it is expected to do, and what functional and non-functional requirements it must satisfy once implemented.

1.2 Scope

The scope of this project spans two phases, with Phase 1 forming the guaranteed and central scope of the project.

Phase 1 covers the fine-tuning and rigorous evaluation of a Sinhala ASR model. Within this scope, the work covers:

- Collection and preprocessing of Sinhala speech-transcript data from multiple public sources (OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala).
- Fine-tuning of the base multilingual Whisper model (Whisper-Small, with Whisper-Base as a fallback) on the collected Sinhala corpora.
- Benchmarking of the fine-tuned model against the base multilingual Whisper model and against an existing published Sinhala Whisper fine-tune, using WER and CER on held-out test data.
- Evaluation of the model's ability to generalise to speech from a source not used in training.
- Error analysis identifying specific, recurring recognition failure patterns.

Phase 2 covers the integration of the fine-tuned model into a downstream accessibility application.

Within this scope, the work covers:

- Development of an education-focused tool that helps students with special needs, such as deaf and hard-of-hearing students and students with writing difficulties, follow lessons and complete their

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

work more easily.

- Implementation of a lightweight inference API (e.g., built with FastAPI or Flask) to serve the fine-tuned model.
- Development of a simple web front-end for capturing microphone audio or uploaded lecture recordings.
- Basic transcript-handling features such as editing and export, so the tool is genuinely usable by students with special needs rather than just a model demo.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ASR	Automatic Speech Recognition.
WER	Word Error Rate, the standard word-level ASR accuracy metric.
CER	Character Error Rate, an ASR accuracy metric computed at the character level.
VAD	Voice Activity Detection.
Whisper	OpenAI's multilingual speech recognition model family, used as the base model for fine-tuning.
SpecAugment	A data augmentation technique masking portions of a spectrogram during training.
DHH	Deaf and hard-of-hearing.
Code-mixing	Interchangeable use of Sinhala and English within the same utterance.
Checkpoint	A saved snapshot of a model's trained parameters.
W&B	Weights & Biases, the experiment-tracking platform used for this project.
FP16	Half precision (mixed precision) training, used to reduce memory and speed up training.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

1.4 References

- [1] O. Kjartansson, S. Sarin, K. Pipatsrisawat, M. Jansche, and L. Ha, "Crowd-Sourced Speech Corpora for Javanese, Sundanese, Sinhala, Nepali, and Bangladeshi Bengali," in Proc. 6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018), 2018.
- [2] OpenSLR, "SLR52 – Large Sinhala ASR Training Data Set." Available: openslr.org/52 (Accessed on 25-Jul-2026).
- [3] Mozilla Common Voice – Sinhala subset. Available: mozilladatacollective.com/datasets/ (Accessed on 25-Jul-2026).
- [4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," OpenAI, 2022.
- [5] Hugging Face, "Fine-Tune Whisper for Multilingual ASR with Transformers." Available: huggingface.co/blog/fine-tune-whisper (Accessed on 25-Jul-2026).
- [6] SPEAK-ASR Hugging Face organization, iterative Whisper LoRA fine-tunes on OpenSLR SLR52 and self-collected YouTube Sinhala audio. Available: huggingface.co/SPEAK-ASR (Accessed on 25-Jul-2026).
- [7] SPEAK-ASR/youtube-sinhala-asr dataset. Available: huggingface.co/datasets/SPEAK-ASR/youtube-sinhala-asr (Accessed on 25-Jul-2026).
- [8] SPEAK-ASR/biz-brains-academy-sinhala dataset. Available: huggingface.co/datasets/SPEAK-ASR/biz-brains-academy-sinhala (Accessed on 25-Jul-2026).
- [9] Lingalingeswaran, "whisper-small-sinhala," Hugging Face model card. Available: huggingface.co/Lingalingeswaran/whisper-small-sinhala (Accessed on 25-Jul-2026).
- [10] Weights & Biases (W&B) documentation. Available: docs.wandb.ai (Accessed on 25-Jul-2026).

1.5 Overview

The remainder of this document is organized as follows: Section 2 describes the product at a high level; Section 3 specifies detailed functional and nonfunctional requirements, and Section 4 provides supporting information, including the project's system level diagram and an open items list.

2. Overall Description

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

SinhaSpeech is a new, self-contained academic research project rather than a follow-on to an existing system. It is positioned against existing Sinhala Whisper fine-tunes, such as those from SPEAK-ASR and independent contributors, as comparison baselines rather than dependencies. The system is best understood as a standalone fine-tuning and evaluation pipeline (Phase 1), extended into a web-based accessibility application (Phase 2) that wraps the Phase 1 model checkpoint behind a lightweight inference API. Phase 2, where pursued, is a new application built entirely on top of this checkpoint rather than an independent system.

The system preprocesses raw Sinhala audio-transcript pairs drawn from multiple public sources, fine-tunes the base multilingual Whisper model across at least three distinct configurations, and benchmarks the resulting model against both base Whisper and existing published Sinhala fine-tunes using WER and CER. Where Phase 2 is pursued, the system additionally serves live or uploaded lecture captioning, spoken-answer transcription for exams and quizzes, and spoken note-taking for self-study, through workflows built on the fine-tuned model.

Target users fall into two distinct groups. Project evaluators, namely the mentor and teaching assistant, along with the development team itself, are technical users primarily concerned with model accuracy, methodology, and reproducibility. The Phase 2 end users, Deaf and hard-of-hearing students and students with writing difficulties, are non-technical users who rely on caption and transcript accuracy, legibility, and ease of correction rather than any advanced or configurable features.

Several practical constraints bound the project's scope. Whisper's feature extractor requires 16kHz, mono, WAV audio input, fixing the target format for preprocessing regardless of the source datasets' original formats. Available GPU memory and session limits on the team's cloud platform, primarily Google Colab Pro with free-tier fallbacks, constrain model choice, which is why Whisper-Base is kept as a fallback to Whisper-Small. Dataset licensing restricts usage to the open research terms under which OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala are published, and Sri Lanka's Personal Data Protection Act further constrains how classroom audio and transcripts are stored within the Phase 2 application.

The project depends on the continued availability of the OpenSLR52, Mozilla Common Voice, and SPEAK-ASR datasets, continued cloud GPU quota on the team's chosen platform, and continued availability of the base multilingual Whisper checkpoint and the Weights & Biases service used for experiment tracking. It also assumes that the datasets used are of sufficient quality and diversity to support meaningful fine-tuning and evaluation, and that existing published Sinhala fine-tunes remain accessible for benchmarking comparisons.

Requirements are organised into functional requirements (preprocessing, fine-tuning, evaluation, captioning, and transcript-editing workflows), reliability requirements (availability, MTBF, MTTR, accuracy, and defect rate), usability requirements (training time, task efficiency, and accessible design), performance and security requirements (response time, throughput, capacity, and data protection), and supportability requirements (coding standards, modularity, and reproducibility). Phase 2 requirements throughout Section 3 are treated as committed project scope, to be pursued once Phase 1 clears its defined WER/CER threshold, rather than as optional or conditional additions.

3. Specific Requirements

3.1 Functionality

This section describes the detailed functional requirements of the SinhaSpeech system, organised by feature area: Preprocessing, Fine-Tuning, Evaluation, the Deaf/hard-of-hearing captioning workflow, the writing-difficulty workflow (covering both exam/quiz answering and self-study note-taking), and shared transcript editing.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Preprocessing, Fine-Tuning, and Evaluation form the guaranteed, core scope of the project. The captioning and writing-difficulty workflows are treated as committed scope, to be pursued once the core pipeline clears its defined WER/CER threshold, with the exception of live microphone captioning, which remains an optional extension pursued only if time permits.

3.1.1 Audio preprocessing pipeline

The system must standardise and clean the raw audio-transcript pairs drawn from all four datasets OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala so that they are consistent and usable for fine-tuning. All input audio is converted to 16kHz, mono, WAV format, and a pretrained Silero VAD model is applied to trim leading and trailing silence from each clip. A light cleanup pass follows, using non-stationary spectral-gating noise reduction via the noisereduce library. Log-Mel spectrogram features are then extracted using Whisper's own feature extractor from the Hugging Face transformers library.

On the transcript side, Sinhala Unicode representation is normalised across all sources, and stray symbols and obvious spelling inconsistencies are corrected during this same normalisation step. Preprocessing statistics clip counts and average duration before and after VAD trimming are logged per dataset, and the pipeline is built so that preprocessing can be run independently on each of the four datasets rather than only as a single combined batch.

3.1.2 Model Fine Tuning

The base multilingual Whisper model is fine-tuned on the preprocessed Sinhala data using at least three distinct configurations. Whisper-Small is used as the primary architecture, with Whisper-Base available as a fallback if hardware constraints make Whisper-Small impractical.

OpenSLR52, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala are pooled together, shuffled, and split into training and validation sets, so the model is exposed to a broad mix of speakers, domains, and recording conditions during training. Mozilla Common Voice Sinhala is withheld entirely from this process and reserved for evaluation, since its different speakers and recording conditions allow generalisation to be measured honestly rather than on data the model has already partially seen.

SpecAugment is applied during training to reduce overfitting, and mixed-precision (FP16) training is used to keep memory and time costs manageable. After every epoch, the model is evaluated on the held-out validation split, computing both WER and CER, and the checkpoint with the best validation WER/CER is selected rather than simply using the last epoch. Training loss, validation loss, learning rate, WER, and CER are logged to Weights & Biases for every run, giving the team a consistent way to compare the three fine-tuning configurations against each other.

3.1.3 Model Evaluation and Benchmarking

The final fine-tuned model is evaluated rigorously against relevant baselines. WER and CER are computed on the held-out test split drawn from the pooled OpenSLR52, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala data, and separately on the fully withheld Mozilla Common Voice Sinhala set, to assess how well the model generalises to speech from a source it never encountered during training. The same metrics are computed for the

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

base multilingual Whisper model and for at least one existing published Sinhala Whisper fine-tune, using identical test sets, so that the comparison is fair. Beyond the aggregate WER/CER figures, a confusion-pattern breakdown is produced to identify frequently mistranscribed phonemes or words, giving insight into why errors occur rather than just how many there are.

VAD precision and recall are also measured against a manually labelled audio subset, to confirm that the silence-trimming step is not inadvertently removing spoken content. A final output report summarising all evaluation results is produced regardless of whether the model clears the accuracy threshold set for the project, since the error analysis itself is treated as a valuable outcome.

3.1.4 Classroom Captioning

This workflow converts a teacher's spoken lecture into readable captions for Deaf and hard-of-hearing students. The core functionality is upload-based: a teacher's lecture, whether recorded as audio or video, is uploaded to the system and converted into a full transcript with readable captions that the student can go through at their own pace.

Live microphone capture with near real-time captioning is treated as an optional extension to be pursued if time permits, rather than a guaranteed part of this workflow, given the additional latency and reliability constraints it introduces.

Captions are displayed with adjustable font size and high contrast so they remain legible under different classroom conditions and for students with varying visual needs. Long lectures are automatically segmented into time-stamped caption chunks rather than presented as one continuous block, making it easier to navigate to a specific part of the lesson, and students are able to download the transcript once it has been generated, so it can be reviewed after class.

3.1.5 Transcript Editing, Export, and Confidence Highlighting

This functionality is shared across both the lecture-captioning and writing-support workflows. The system renders per-word or per-segment confidence scores and visually flags any word that falls below a configurable confidence threshold, helping the student focus their review on the parts of the transcript most likely to be wrong, whether that transcript is an exam answer, a set of self-study notes, or a lecture caption. Users can edit the transcript inline without losing the underlying audio-to-text alignment, so corrections do not break the connection between the text and the corresponding moment in the recording. Transcripts can be exported as .txt, .docx, or .pdf depending on the context, and the original recording is retained alongside its transcript, so the source audio remains available for later reference or re-transcription if needed.

3.1.6 Spoken Answer Transcription

Supporting students for whom handwriting or typing is slow or effortful. Teachers can make quizzes and exams available through the platform, which students answer by speaking rather than writing; students can also use the same workflow independently for self-study, working through practice papers, mock exams, or quizzes on their own.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

The system records audio directly or accepts an uploaded clip of the spoken answer, transcribes it into editable text, and allows the student to review and correct that text before submission. Low-confidence words are visually highlighted throughout, drawing the student's attention to the parts of the answer most likely to need correction. Once the student is satisfied with the corrected text, it can be exported as a .txt or .docx file for submission.

3.1.7 Self-Study Note-Taking

This workflow converts a student's spoken notes into editable text to support self-study. Rather than answering a set question, the student speaks freely to capture ideas, summaries, or study points, and the system transcribes this into text that can be reviewed, corrected, and organised afterwards. The system records audio directly or accepts an uploaded clip of spoken notes, transcribes it into editable text, and allows the student to revise the content before saving it. As with the exam workflow, low-confidence words are visually highlighted so the student can focus their review on the parts of the transcript most likely to contain errors. The finalised notes can then be exported as a .txt or .docx file for later use.

3.2 Usability

This section includes requirements affecting usability for both the development team and end users. Since a new DHH or writing-difficulty student may have no prior experience with the tool, the core workflow upload or record, then view the transcript must be usable without training, guided only by a brief on-screen prompt. Captions and transcripts must be presented clearly enough that users can act on them within seconds of generation rather than minutes, since both target use cases (following a live lesson, answering a timed exam question) are time-sensitive.

Editing a transcript should also be straightforward and low-friction: low-confidence words are visually flagged so users can quickly locate what needs review, and inline corrections can be made directly without disrupting the flow of reading or reviewing the rest of the text. The interface should also follow familiar, standard web-application design patterns and accessible design practices, including adjustable font size, sufficient colour contrast, and full keyboard navigability, so the application remains usable by students with varying visual, motor, and hearing needs.

3.3 Reliability

Availability

The application shall be available and functional at least 95% of the time during scheduled classroom use. Maintenance windows should be scheduled outside these hours to minimise disruption to teaching and self-study sessions.

Mean Time Between Failures (MTBF)

The application should be designed to achieve a mean time between failures of at least 100 hours of continuous operation, ensuring stable performance during captioning sessions and transcript-editing workflows.

Mean Time to Repair (MTTR)

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

In the event of a failure, the application should be recoverable and operational again within an average of 2 hours after the issue is identified and diagnosed.

Accuracy

The fine-tuned model must achieve WER/CER measurably better than base Whisper, and lower than or comparable to existing published Sinhala fine-tunes, on the held-out test sets defined in Section 3.1.

Defect Rate

The codebase should aim for no more than 2 critical bugs per thousand lines of code (KLOC) at release, where a critical bug is defined as one causing loss of a transcript, incorrect confidence highlighting affecting all transcripts, or an inability to complete a captioning or transcription session.

3.4 Performance and Security

The system's performance and security characteristics for the application are outlined below.

Response time: Uploaded-file transcription shall complete within a reasonable multiple of the audio's duration.

Throughput: The application should support at least one concurrent transcription session per classroom deployment, given that the primary interaction mode is upload-based rather than live.

Capacity: The application shall handle uploaded audio and video files up to a reasonable maximum size and duration without failure.

Degradation modes: Live captioning, treated as an optional extension rather than core functionality, should fall back to processing incoming audio in short delayed batches if it cannot keep pace in real time, rather than dropping audio outright.

Resource utilisation: The application shall operate within the resource limits of the hosting platform used for deployment (e.g., a free-tier or low-cost cloud host).

Data privacy and security: The system shall store classroom audio, transcripts, and any associated student information in a secure database, with access restricted to authenticated users and limited to what is necessary for the transcription and review workflow. Stored data shall be handled consistent with Sri Lanka's Personal Data Protection Act, including safeguards such as access control and encryption of data at rest where feasible. API keys and credentials for third-party services shall not be hard-coded in source code, and shall instead be managed through environment variables or a secrets manager

3.5 Supportability

Coding standards: The system shall adhere to industry-recognized coding standards, ensuring consistent formatting and readability across the codebase. Following standardized code practices enhances maintainability and eases future debugging and upgrades.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Naming conventions: Consistent naming conventions shall be followed for variables, functions, classes, and files to improve clarity and reduce onboarding time for new developers. For example, camelCase or snake_case shall be used uniformly across different modules.

Common libraries: Where possible, the system shall utilise widely adopted and actively maintained open-source libraries (e.g., Hugging Face Transformers/Datasets, PyTorch, pandas, scikit-learn) to minimise the risk of deprecated or unsupported dependencies.

Modular Architecture: The system shall be structured into clearly separated, independent modules preprocessing, fine-tuning, evaluation, and the Phase 2 application so that each component can be developed, tested, modified, or replaced without requiring changes to the others.

3.6 Design Constraints

Standards Compliance

Data formats (CSV, JSON) shall use UTF-8 encoding to correctly support Sinhala Unicode throughout preprocessing, training, and transcript storage. The system must adhere to recognised conventions for ASR data preprocessing and fine-tuning (e.g., the Hugging Face Transformers/Datasets workflow), standard evaluation metrics (WER, CER), and, for Phase 2, RESTful API design principles with JSON as the interaction format.

Hardware Limitations

Available GPU memory and session limits on the team's cloud platform (e.g., Google Colab Pro, with Google Colab and Kaggle Notebooks as free-tier fallbacks) constrain model size (Whisper-Small primary, Whisper-Base fallback), batch size, and training duration. No on-premises hardware is used; all fine-tuning and evaluation runs within the resource limits of the chosen cloud environment.

Software Stack

The system must be implemented primarily in Python, using Hugging Face Transformers and Datasets for model loading, fine-tuning, and evaluation, PyTorch as the underlying deep learning framework, and Silero VAD and noisereduce for audio preprocessing. The Phase 2 backend uses a lightweight Python web framework (e.g., FastAPI or Flask) to serve the fine-tuned model behind an inference API.

Development Tools and Environment

Development must be conducted in environments that support version control (Git) and notebook-based prototyping (Jupyter/Colab), consistent with the team's use of Google Colab Pro for training. Use of the base Whisper checkpoint and any pre-trained components must comply with their respective licensing terms and be version-tracked alongside experiment logs in Weights & Biases.

Architectural Constraints

The system follows a modular, component-based layered architecture that separates preprocessing, fine-tuning, evaluation, and the Phase 2 application (front end, inference API, database) into independently testable components. Integration between the Phase 2 front end, inference API, and database must occur through well-defined interfaces, consistent with the Software Interfaces described in Section 3.2.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

3.7 On-line User Documentation and Help System Requirements

Technical report: Documents each of the three fine-tuning configurations in detail, including hyperparameters, data mix, and results.

Reproducibility guide: Covers the preprocessing, training, and evaluation steps needed to reproduce the project's methodology.

Model card: Summarises the fine-tuned model's architecture, training data, and benchmark results.

In-app quick-start guide : Walks a first-time user through the core workflow relevant to them, lecture upload and captioning, spoken exam/quiz answering, or self-study note-taking, made available in Sinhala as well as English.

3.8 Purchased Components

The system relies primarily on open-source tools and libraries, including Hugging Face Transformers and Datasets, PyTorch, Silero VAD, and noisereduce, all of which are freely available and require no licensing cost. Weights & Biases is used for experiment tracking under its free tier, which is sufficient for the scale of this project. The main purchased component is rented cloud GPU compute, such as Google Colab Pro, used for model fine-tuning and evaluation at a low monthly subscription cost, with free-tier platforms (Google Colab, Kaggle Notebooks) available as a fallback if budget is constrained. No other commercially licensed components are currently planned

3.9 Interfaces

This section defines the interfaces that must be supported by the application, at a level of detail sufficient to develop and verify against them. Screen mockups are not included at this stage, as the Phase 2 interface has not yet been implemented; the subsections below describe required functionality and content instead.

3.9.1 User Interfaces

Lecture Upload and Captioning Screen (DHH Workflow):

Allows a teacher or student to upload an audio or video file of a lecture. Once processing completes, the interface displays the generated transcript as time-stamped caption chunks, with controls for adjusting font size and toggling high-contrast display. Users can download the finished transcript from this screen.

Spoken Answer Screen (Exam/Quiz Workflow):

Allows a student to record audio directly through the browser or upload a pre-recorded clip in response to an exam or quiz question. Once transcribed, the answer is shown in an editable text field with low-confidence words visually flagged, alongside options to correct the text and submit or export it.

Self-Study Notes Screen:

Allows a student to record or upload spoken notes outside of any assigned exam or quiz context. The resulting transcript is shown in the same editable format as the answer screen, allowing the student to revise and organise their notes before saving or exporting.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Shared Transcript Editor Component:

Used across all three workflows above, this component renders the transcript text with inline editing support, per-word confidence highlighting, and export options (.txt, .docx, .pdf), while preserving the underlying audio-to-text alignment as edits are made.

Teacher/Quiz Management Screen:

Allows a teacher to create and make available quizzes or exam questions that students answer using the Spoken Answer workflow.

3.9.2 Hardware Interfaces

The software is intended to run on general-purpose computing devices, including laptops, desktops, or cloud-based virtual machines. It does not require any dedicated hardware interfaces or physical connections to external devices. However, the system must be compatible with standard I/O interfaces such as keyboards and displays for basic user interaction. There are no specific physical addresses involved, and the application does not interact with hardware at a low level. The system should perform reliably on student laptops with a standard processor and standard storage capacity. If deployed in environments handling large datasets or requiring real-time responses, enhanced hardware such as SSD storage and GPU acceleration may improve performance but are not mandatory.

3.9.3 Software Interfaces

The system will interact with several external and internal software components:

- Whisper Model and Hugging Face Transformers/Datasets
 - Interfaces through Python APIs within preprocessing, training, and evaluation scripts.
 - Used for loading the base multilingual Whisper checkpoint, tokenising transcripts, and running fine-tuning and inference.
- Weights & Biases (W&B) API
 - Protocol: HTTPS
 - Format: JSON requests and responses
 - Authentication: API key-based
 - Used for logging and visualising training loss, validation loss, learning rate, WER, and CER during fine-tuning runs.
- Preprocessing Libraries (Silero VAD, noisereduce)
 - Interfaces through Python APIs within the preprocessing pipeline.
 - Used for silence trimming and light noise reduction on raw audio before feature extraction.
- Inference API (FastAPI/Flask)
 - Protocol: HTTPS
 - Format: JSON requests and responses
 - Serves the fine-tuned Whisper model to the Phase 2 front end, accepting audio input and returning transcripts with confidence scores.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

- Data I/O Interfaces
 - Supports reading and writing data in WAV (audio), JSON, and CSV formats for datasets, logs, and evaluation results.
 - Supports exporting finalised transcripts in .txt, .docx, and .pdf formats for end users.

3.9.4 Communications Interfaces

The system will rely on standard internet-based communication protocols to interact between its components. The Phase 2 front end communicates with the inference API over HTTPS, using JSON-formatted requests and responses, ensuring secure transmission of audio uploads, transcripts, and confidence data between the client and server.

Asynchronous HTTP requests may be used for the front end to poll or receive transcription results without blocking the user interface, particularly for longer uploaded files. The application's database connections are made over a secure, authenticated connection appropriate to the chosen hosting platform.

Experiment tracking and logging to Weights & Biases during Phase 1 also occurs over HTTPS. The system does not require low-level serial or parallel communication but must maintain stable connectivity under typical broadband and classroom Wi-Fi conditions, and all communication must comply with common network and firewall settings to ensure compatibility during deployment and classroom use.

3.10 Licensing, Legal, Copyright, and Other Notices

SinhaSpeech is an academic research project, and no proprietary licensing enforcement is required. The system relies on open-source libraries (Hugging Face Transformers/Datasets, PyTorch, Silero VAD, noisereduce) and open-research datasets (OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, Biz-Brains-Academy-Sinhala), all used strictly within the terms of their respective licences; no datasets are redistributed beyond those terms. Existing published Sinhala Whisper fine-tunes (e.g., SPEAK-ASR models, whisper-small-sinhala) are used only as benchmark comparisons and are properly attributed in the technical report and references. The base multilingual Whisper model is used under its original OpenAI licence, and any fine-tuned checkpoint produced by this project inherits the applicable terms of that licence. No trademarked names, logos, or wordmarks belonging to any third party are used in a way that implies endorsement or affiliation. As the resulting model and application are intended for academic evaluation, no warranty of fitness for production or clinical use is provided.

3.11 Applicable Standards

Data encoding:

All text data (transcripts, exported files) must conform to UTF-8 Unicode encoding to correctly represent Sinhala script.

Audio format:

All processed audio must conform to the 16kHz, mono, WAV format expected by Whisper's feature extractor.

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

Evaluation metrics:

Model accuracy is reported using the standard ASR evaluation metrics Word Error Rate (WER) and Character Error Rate (CER).

API design:

Phase 2 backend APIs follow RESTful design principles, using JSON as the interaction format.

Accessibility:

The Phase 2 interface follows common web accessibility practices, including sufficient colour contrast, adjustable font size, and keyboard navigability, informed by WCAG 2.1 guidance.

Data protection:

Storage and handling of classroom audio, transcripts, and student data in Phase 2 must comply with Sri Lanka's Personal Data Protection Act.

4. Supporting Information

This SRS includes a table of contents to aid navigation across sections. No index or appendices are included at this stage, as Phase 2 interface prototypes and use-case storyboards have not yet been developed; these may be added as appendices in a future revision once available. Should appendices be added later, they will be explicitly marked as either part of the formal requirements or as supplementary, non-binding material, and this distinction will be stated clearly at the start of each appendix.

4.1 References

- [1] O. Kjartansson, S. Sarin, K. Pipatsrisawat, M. Jansche, and L. Ha, "Crowd-Sourced Speech Corpora for Javanese, Sundanese, Sinhala, Nepali, and Bangladeshi Bengali," in *Proc. 6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018)*, 2018.
- [2] OpenSLR, "SLR52 – Large Sinhala ASR Training Data Set." [Online]. Available: <https://openslr.org/52> (Accessed on 25-Jul-2026).
- [3] Mozilla, "Common Voice – Sinhala subset." [Online]. Available: <https://mozilladatacollective.com/datasets/> (Accessed on 25-Jul-2026).
- [4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," OpenAI, 2022.
- [5] Hugging Face, "Fine-Tune Whisper for Multilingual ASR with Transformers." [Online]. Available: <https://huggingface.co/blog/fine-tune-whisper> (Accessed on 25-Jul-2026).
- [6] SPEAK-ASR, Hugging Face organization page. [Online]. Available: <https://huggingface.co/SPEAK-ASR> (Accessed on 25-Jul-2026).
- [7] SPEAK-ASR, "youtube-sinhala-asr" dataset. [Online]. Available:

SinhaSpeech	Version 1.0
Software Requirement Specification Document	30-07-2026

<https://huggingface.co/datasets/SPEAK-ASR/youtube-sinhala-asr> (Accessed on 25-Jul-2026).

[8] SPEAK-ASR, "biz-brains-academy-sinhala" dataset. [Online]. Available: <https://huggingface.co/datasets/SPEAK-ASR/biz-brains-academy-sinhala> (Accessed on 25-Jul-2026).

[9] Lingalingeswaran, "whisper-small-sinhala," Hugging Face model card. [Online]. Available: <https://huggingface.co/Lingalingeswaran/whisper-small-sinhala> (Accessed on 25-Jul-2026).

[10] Weights & Biases (W&B) documentation. [Online]. Available: <https://docs.wandb.ai> (Accessed on 25-Jul-2026).

[11] Sri Lanka, Personal Data Protection Act, No. 9 of 2022.

[12] Silero Team, "Silero VAD: pre-trained enterprise-grade Voice Activity Detector." [Online]. Available: <https://github.com/snakers4/silero-vad> (Accessed on 25-Jul-2026).

[13] T. Sainburg, "noisereduce: Noise reduction in python using spectral gating." [Online]. Available: <https://github.com/timsainb/noisereduce> (Accessed on 25-Jul-2026).